



DEPARTMENT OF COMPUTER SCIENCE UNIVERSITY OF GWADAR

CS-454- Deep Learning

Instructor Information

Name	Hafeezullah
E-mail	hafeezullah@ug.edu.pk
Google Chat	hafeezullah@ug.edu.pk
LinkedIn Profile	www.linkedin.com/in/hafeez-ullah-7a5328353
Personal Website	https://hafeezug.github.io/
Research Lab	Artificial Intelligence and Data Innovation Research Lab (AIDIRL) https://aidirl.github.io/
GitHub	https://github.com/hafeezug
Hugging Face	https://huggingface.co/hafeez007
Office Hours	Mon – Fri 8:30 am - 4:30 pm

Course Information

Program	BSIT/BSDS
Session	S2024-S2027
Offering Semester	6 th Fall 2026
Course Title	Deep Learning
Course Code	CS-454- CS-454L
Course Category	IT/DS Elective
Credit Hours	2 (3+1)
Contact Hours	5 (2+3)
Prerequisites	Artificial Intelligence/Introduction to Data Science
Total Teaching Weeks	16
Total Lectures	16
No. of Labs	16
Duration of a Lecture	2 Hours
Duration of a Lab	3 Hours
Lecture Days	
Lab Days	

Grading Policy

Theory (100 Marks)		
Task	Quantity	Weightage
Sessional		30%
Assignments	3	9%
Quizzes	3(2 best will be counted)	6%
Term Project(Prototype, Presentation, Report)	1	15%
Mid Term		30%

Final Term		40%
Lab (50 Marks)		
Sessional		25%
Assignments	2	6%
Lab Tasks	16	8%
Open Ended Lab	2	6%
Lab Manual	1	5%
Final Term		25%

Course Introduction

Deep Learning is the driving force behind modern AI breakthroughs. This course covers the theoretical foundations and practical implementation of deep neural networks, including feedforward networks, convolutional neural networks (CNNs), recurrent networks (RNNs/LSTMs), transformer-based architectures, and generative models. Students will train, evaluate, and fine-tune models on real-world datasets using PyTorch and TensorFlow

Course Learning Outcomes (CLOs)

CLO #	Course Learning Outcome (CLO)	Bloom's Taxonomy Level	Mapped PLO(s)
CLO-1	Explain the architecture, training mechanisms, and mathematical foundations of deep learning models including CNNs, RNNs, LSTMs, and transformers.	C2 – Understand PLO-2	PLO-2
CLO-2	Implement appropriate deep learning models for real-world problems in computer vision, NLP, and time-series analysis using modern frameworks.	C3 – Apply	PLO-4, PLO-5
CLO-3	Analyze and compare different deep learning architectures in terms of performance, computational cost, and suitability for a given task.	C4 – Analyze	PLO-3, PLO-5
CLO-4	Design and fine-tune deep learning pipelines including data augmentation, regularization, hyperparameter tuning, and transfer learning strategies.	C6 – Create	PLO-4, PLO-5
CLO-5	Evaluate the ethical implications and societal impact of deploying deep learning systems.	C5 – Evaluate	PLO-8, PLO-9

Course Outline and Weekly Lecture Breakdown (Theory)

WEEK	Topic
1	Logistics, Introduction to Deep Learning & Linear Algebra Essentials Course Logistics, policies, grading criteria, and learning roadmap.

	AI vs. Machine Learning vs. Deep Learning paradigm shift; Key components of deep learning: Data, Models, Loss/Objective Functions, Optimization Algorithms; Real-world deep learning applications. Linear Algebra for Deep Learning: Scalars, Vectors, Matrices, and Tensors; Dot Products, Matrix Operations, Norms, Tensor ranks and shapes; Vector spaces and geometric orientations.
2	Linear Models to Classification: Linear & Softmax Regression Linear Regression: Mathematical formulation, Single-Layer Neural Network representation, Feature weighting and bias parameters, Mean Squared Error (MSE) objective function. Optimization Foundations: Gradient Descent (GD), Learning Rate dynamics, Mini-Batch Stochastic Gradient Descent (SGD) algorithm. Softmax Regression: Regression vs. Multiclass Classification, Network architecture, One-Hot encoding, Softmax activation function, Cross-Entropy Loss formulation; Evaluation on classification tasks (MNIST, ImageNet).
3	Multilayer Perceptrons (MLPs), Non-Linearities & Backpropagation Mechanics Multilayer Perceptrons: Affine transformations and limitations of linear models, Monotonicity vs. Non-linearity, Hidden layer representations, Universal Approximation Theorem. Activation Functions: Sigmoid, Hyperbolic Tangent (Tanh), Rectified Linear Unit (ReLU), and LeakyReLU. Forward and Backward Propagation: Computational Graphs, Forward pass intermediate variable calculations, Reverse-mode automatic differentiation, Chain Rule of calculus, and parameter gradient computations.
4	Numerical Stability, Weight Initialization & Generalization in Deep Learning Numerical Stability: Vanishing and Exploding Gradients in deep architectures; Matrix multiplication dynamics across deep layers; Breaking symmetry in neural weights. Weight Initialization Strategies: Pitfalls of constant/random initialization, Xavier/Glorot initialization, and He/Kaiming initialization. Generalization Theory: Training error vs. Generalization error, Model capacity and complexity, Overfitting vs. Underfitting diagnostics, Validation datasets, and K-fold cross-validation strategies.
5	Regularization Techniques for Deep Neural Networks Overfitting Mitigation: Weight Decay (L2 Regularization) and parameter shrinkage mechanisms; L1 Regularization and sparsity induction. Practical Regularization Strategies: Regularized loss optimization, Early Stopping criteria and validation monitoring; Dropout mechanics, Inverted Dropout during training vs. inference scaling. Hyperparameter Tuning: Learning rate schedules, batch size effects, and regularization strength lambda.
6	Convolutional Neural Networks I: Foundations & Spatial Feature Extraction

	Computer Vision Overview: Image Classification, Object Localization, Object Detection, and Semantic Segmentation tasks; Digital image tensors (Height, Width, RGB Channels). Convolution Operations: 2D Cross-Correlation/Convolution, Kernels and Filters, Strides, Padding, Output spatial dimension formulas, Receptive Field concept. Pooling and Downsampling: Max Pooling and Average Pooling layers; Building a baseline Convolutional Neural Network architecture with convolutional, pooling, and fully connected layers.
7	Convolutional Neural Networks II: Deep Vision Backbones & Transfer Learning Evolution of Deep CNNs: LeNet-5, AlexNet, VGG-16/19, and GoogLeNet (Inception modules). Residual Learning: Degradation problem in deep networks, Residual blocks with identity skip connections (ResNet-18/50/101), Batch Normalization (nn.BatchNorm2d). Transfer Learning: Pretrained vision backbones (ImageNet), Feature extraction vs. Full model Fine-Tuning, Domain adaptation for custom vision datasets.
8	Midterm Examination & Sequential Data / Language Modeling Foundations Midterm Assessment: Comprehensive evaluation of theoretical and mathematical foundations (Weeks 1-7). Foundations of Sequence Modeling: Sequential data characteristics, Markov chain assumptions, N-gram statistical language models (Unigrams, Bigrams, Trigrams), Language model evaluation via Perplexity. Text Preprocessing Pipeline: Text normalization, Vocabulary construction, Tokenization hierarchies (Character-level, Word-level, Byte-Pair Encoding (BPE)).
9	Recurrent Neural Networks (RNNs) & Classical Text Vectorization Recurrent Neural Networks: Recurrent architecture, Recurrent hidden state update dynamics, Backpropagation Through Time (BPTT), Gradient vanishing and exploding in recurrent loops. Gated Sequence Architectures Overview: Long Short-Term Memory (LSTM) gates (Forget, Input, Output) and Gated Recurrent Units (GRU). Classical Text Representation: Bag-of-Words (BoW) models, Count Vectorization, Term Frequency-Inverse Document Frequency (TF-IDF), Sparse vs. Dense feature spaces for NLP.
10	Dense Word Representations & Word Embeddings From Sparse to Dense Representations: Limitations of One-Hot and Bag-of-Words representations; The Distributional Hypothesis ('Words in similar contexts have similar meanings'). Word2Vec Framework: Continuous Bag of Words (CBOW) and Continuous Skip-Gram models; Negative Sampling and Hierarchical Softmax optimizations. Embedding Geometry: Semantic vector spaces, Word analogy arithmetic (King - Man + Woman = Queen), Cosine similarity metrics, and PyTorch Embedding layers (nn.Embedding).
11	Transformers I: Self-Attention Mechanism & Architecture

	Limitations of Recurrence: Sequential bottlenecks, memory constraints, and lack of parallelization in RNNs/LSTMs. Attention Mechanics: Dot-Product Attention, Query (Q), Key (K), and Value (V) linear projections; Scaled Dot-Product Attention formula: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$. Multi-Head Self-Attention: Splitting subspace projections, Multi-Head Attention blocks, Sinusoidal and Learnable Positional Encodings, Transformer Encoder Block architecture (LayerNorm, Residual Add, Feed-Forward MLP).
12	Transformers II: Full Architecture & Pretrained Model Families Complete Transformer Architecture: Autoregressive Decoder, Masked Multi-Head Self-Attention (causal look-ahead mask), Encoder-Decoder Cross-Attention mechanism. Pretraining Paradigms in NLP: Masked Language Modeling & Bidirectional Encoders (BERT, RoBERTa); Autoregressive Causal Language Modeling (GPT series); Sequence-to-Sequence Modeling (T5, BART). Comparative evaluation of Encoder-only, Decoder-only, and Seq2Seq frameworks.
13	Large Language Models I: Autoregressive Architectures & Generation Dynamics Decoder-Only LLMs: Evolution from GPT-2/3 to modern LLM architectures (Llama, Mistral), Causal self-attention masking, Expanding context windows (RoPE, FlashAttention). Autoregressive Text Generation: Next-token probability distribution modeling, Logits computation, and Temperature scaling. Decoding & Sampling Strategies: Greedy search, Beam search, Top-k sampling, Top-p (Nucleus) sampling, Repetition penalties, and Hallucination mitigation in generation.
14	Large Language Models II: Scaling Laws, In-Context Learning & Prompting Scaling Laws: Empirical scaling laws for compute budget, parameter count, and dataset size (Chinchilla scaling); Emergent capabilities in large-scale models. In-Context Learning & Prompt Engineering: Zero-Shot, One-Shot, and Few-Shot prompting paradigms; Chain-of-Thought (CoT) and Tree-of-Thoughts reasoning. Augmenting LLMs: Retrieval-Augmented Generation (RAG) architectural foundations, Vector databases, Embedding-based semantic search, and Context injection.
15	Fine-Tuning Large Models & Parameter-Efficient Fine-Tuning (PEFT) Fine-Tuning Strategies: Full Model Fine-Tuning challenges, Catastrophic Forgetting, Compute and memory bottlenecks. Parameter-Efficient Fine-Tuning (PEFT): Adapter layers, Prefix Tuning, Prompt Tuning, Low-Rank Adaptation (LoRA) mathematics and rank decomposition, QLoRA (4-bit NormalFloat quantization). Instruction Tuning & Alignment: Supervised Instruction Fine-Tuning (SFT), Dataset formatting (Alpaca/ShareGPT), Preference alignment overview (RLHF with Reward Modeling, Direct Preference Optimization - DPO).
16	Diffusion Models for Generative AI & Course Synthesis

	Generative AI Landscape: Autoregressive models vs. VAEs vs. GANs vs. Diffusion Models. Diffusion Mechanics: Forward Noising Process (Markov chain adding Gaussian noise with variance schedule β_t); Reverse Denoising Process (Learned Score Network / U-Net with timestep conditioning). Text-to-Image Synthesis: Conditioning U-Net with text embeddings (CLIP), Latent Diffusion Models (Stable Diffusion architecture); Course wrap-up, Final Capstone Project evaluation and term project defense.
--	--

Laboratory		
Lab	Activity	Colab Link
1	PyTorch Tensor Manipulation, Vector Math & Automatic Differentiation Setup Google Colab and PyTorch environment; allocate CPU and CUDA GPU devices. Perform tensor creation, slicing, reshaping, broadcasting, and matrix operations ($A \cdot B$, A^T, trace). Implement automatic differentiation using <code>torch.autograd</code>; compute first and second-order gradients on multi-variable polynomial computational graphs. Dataset: Synthetic 2D/3D vectors and polynomial mathematical functions.	
2	Linear and Softmax Regression from Scratch & PyTorch nn.Module Implement Linear Regression from scratch with manual gradient updates and MSE loss. Re-implement with PyTorch <code>nn.Linear</code>, <code>nn.MSELoss</code>, and <code>optim.SGD</code>. Build Softmax Regression for multiclass classification using <code>nn.CrossEntropyLoss</code>; train and evaluate classification accuracy. Dataset: California Housing Dataset (Regression) & MNIST Hand-written Digits (Classification).	
3	Multilayer Perceptrons (MLPs) & Custom Backpropagation Engine Implement a 3-layer MLP from scratch with forward pass, cross-entropy loss, and manual analytical backpropagation. Rebuild the MLP in PyTorch using <code>nn.Sequential</code> and custom <code>nn.Module</code>.	

	Benchmark non-linear activation functions (Sigmoid, Tanh, ReLU, LeakyReLU, ELU) on decision boundary separation and training speed. Dataset: Fashion-MNIST (60,000 28x28 grayscale images across 10 clothing classes).	
4	Diagnostic Training: Weight Initialization, Regularization & Overfitting Implement and evaluate weight initialization strategies: Zero initialization, Random Normal, Xavier/Glorot, and He/Kaiming initialization. Implement L2 Weight Decay in optimizer and add nn.Dropout layers. Implement Early Stopping based on validation loss; plot train vs. validation loss curves to diagnose overfitting/underfitting regimes. Dataset: Breast Cancer Wisconsin (Diagnostic) Dataset / Bank Marketing Dataset.	
5	Convolutional Neural Networks (CNNs) & Feature Map Visualizations Implement 2D Convolution, Max-Pooling, and Batch Normalization layers in PyTorch (nn.Conv2d, nn.MaxPool2d, nn.BatchNorm2d). Construct a custom deep CNN architecture with convolutional feature extractors and linear classification heads. Visualize learned convolution filters and intermediate activation feature maps for input images. Dataset: CIFAR-10 Dataset (60,000 32x32 color images across 10 classes).	
6	Deep Residual Networks (ResNet) & Computer Vision Transfer Learning Implement custom Residual Blocks with skip connections (shortcut path and conv path) in PyTorch. Load pretrained vision backbones (ResNet-18 / ResNet-50) from torchvision.models. Apply data augmentation pipelines (torchvision.transforms); freeze convolutional backbone weights and fine-tune classifier heads for custom image classification. Dataset: Oxford-IIIT Pet Dataset (37 pet breeds) / Dogs vs. Cats Dataset.	
7	Object Detection Metrics & Semantic Segmentation with Mini U-Net	

	Implement computer vision evaluation metrics: Intersection over Union (IoU), Precision-Recall curves, and Non-Maximum Suppression (NMS). Construct a lightweight U-Net architecture featuring contracting encoder blocks, bottleneck, and expanding decoder blocks with skip connections. Train the segmentation model for pixel-level binary mask segmentation. Dataset: Oxford-IIIT Pet Segmentation Masks / Pascal VOC Mini Subset.	
8	Text Preprocessing, Tokenization & Bag-of-Words / TF-IDF Classification Build a complete text preprocessing pipeline: regex cleaning, lowercase normalization, stopwords filtering, and n-gram extraction. Implement Byte-Pair Encoding (BPE) and WordPiece tokenization using Hugging Face tokenizers. Construct CountVectorizer and TF-IDF matrix representations; train a PyTorch neural classifier for sentiment analysis. Dataset: IMDB Large Movie Review Dataset (50,000 positive/negative sentiment reviews).	
9	Recurrent Neural Networks (RNNs, LSTMs & GRUs) for Sequence Modeling Implement Character-level RNN from scratch in PyTorch for sequence generation. Build LSTM (nn.LSTM) and GRU (nn.GRU) models to mitigate vanishing gradients in long sequences. Train language models on text corpora, evaluate model perplexity on test sequences, and generate sample text with temperature sampling. Dataset: H.G. Wells' The Time Machine / Shakespeare Play Corpus.	
10	Word Embeddings: Word2Vec (Skip-Gram with Negative Sampling) & nn.Embedding Implement target-context word pair extraction from text corpus with sliding context windows. Train a Skip-Gram model with Negative Sampling in PyTorch using nn.Embedding lookup tables. Extract learned dense embedding weights, compute cosine similarity between word vectors, and visualize 2D word clusters using t-SNE / PCA. Dataset: WikiText-2 Dataset / Text8 Wikipedia Corpus.	

11	Implementing Scaled Dot-Product & Multi-Head Self-Attention from Scratch Implement Query, Key, and Value (Q, K, V) projection layers from scratch in PyTorch. Compute Scaled Dot-Product Attention matrix and apply attention masking. Build Multi-Head Attention layer with tensor splitting, scaled dot-product computation, head concatenation, and final linear projection. Implement Sinusoidal Positional Encoding and assemble a complete Transformer Encoder block. Dataset: Synthetic Sequence-to-Sequence Translation Data / IMDB Sentiment Subset.	
12	Fine-Tuning Pretrained Transformer Models (BERT & GPT-2) Load pretrained BERT (AutoModelForSequenceClassification) and tokenizer (AutoTokenizer) using Hugging Face transformers. Fine-tune BERT on downstream NLP classification tasks with AdamW optimizer and linear learning rate scheduler. Load pretrained GPT-2 (AutoModelForCausalLM); execute autoregressive text generation using Greedy, Top-k, Top-p (Nucleus), and Temperature sampling. Dataset: GLUE SST-2 (Stanford Sentiment Treebank) / AG News Topic Dataset.	
13	LLM In-Context Prompting & Building a Retrieval-Augmented Generation (RAG) System Implement structured prompting techniques: Zero-shot, Few-shot demonstration prompting, and Chain-of-Thought (CoT) reasoning. Build a local Retrieval-Augmented Generation (RAG) system: parse documents, chunk text recursively, compute sentence embeddings (sentence-transformers), store in ChromaDB / FAISS vector database. Query vector store, construct context-augmented prompts, and generate factually grounded answers using an LLM. Dataset: SQuAD 2.0 (Stanford Question Answering Dataset) / Academic Course Notes.	
14	Parameter-Efficient Fine-Tuning (PEFT) with LoRA and QLoRA Load an open-source LLM (Llama-3-8B-Instruct / Mistral-7B / SmolLM) in 4-bit precision using bitsandbytes (QLoRA).	

	Configure and inject Low-Rank Adaptation (LoRA) adapters with Hugging Face peft (r=8, lora_alpha=16, target_modules=['q_proj', 'v_proj']). Fine-tune the quantized LLM on custom instruction-following dataset using trl.SFTTrainer; evaluate inference quality before and after fine-tuning. Dataset: Stanford Alpaca Instruction Dataset / Databricks Dolly-15k Subset.	
15	Diffusion Models: Forward Noise Schedules & Text-to-Image Generation Implement DDPM forward diffusion process: add linear and cosine scheduled Gaussian noise to images over T=1000 timesteps. Train a toy U-Net diffusion model on 2D images to predict injected noise epsilon_theta. Execute text-conditioned image generation using Hugging Face diffusers (Stable Diffusion Pipeline); visualize intermediate denoising trajectory steps across diffusion timesteps. Dataset: MNIST / Fashion-MNIST (Toy DDPM Training) & Descriptive Text Prompts (Stable Diffusion).	
16	Deep Learning Model Optimization, ONNX Export & Interactive UI Deployment Export trained PyTorch models to ONNX and TorchScript formats; optimize graphs for inference speed. Benchmark inference latency, throughput, memory footprint, and CPU vs. GPU execution. Build an interactive web demo using Gradio / Streamlit; deploy the working model demo publicly to Hugging Face Spaces. Final Term Project Demonstration and Lab Portfolio Review. Dataset: Capstone Term Project Dataset (Vision / NLP / Multimodal).	
Textbooks		
1. Ian Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning (http://www.deeplearningbook.org/) 2. Françoise Chollet, Deep learning with python, ISBN-10: 9781617294433, 2017 3. Omar Sanseviero, Pedro Cuenca, Apolinário Passos & Jonathan Whitaker, Hands on Generative AI with Transformers and Diffusion Models, O'Reilly Media, Inc., 2024 4. D Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola, Dive into Deep Learning Eli Stevens, Luca Antiga, And Thomas Viehmann, Deep Learning with Pytorch, Manning Publications Co.		

Supplementary Resources

- Hugging Face Course: <https://huggingface.co/course>

Platforms:

- Google Colab (free GPU)
- Kaggle Notebooks
- HuggingFace Spaces
- AWS SageMaker (education tier)